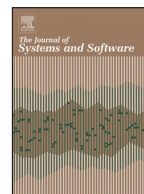




Contents lists available at ScienceDirect

The Journal of Systems and Software

journal homepage: www.elsevier.com/locate/jss

Going with the flow: An activity theory analysis of flow techniques in software development

Denis Dennehy*, Kieran Conboy

J.E. Cairnes School of Business & Economics, National University of Ireland Galway, Upper Newcastle Road, Galway city, Ireland

ARTICLE INFO

Article history:

Received 21 March 2016

Revised 29 September 2016

Accepted 4 October 2016

Available online xxx

Keywords:

Activity theory

Lean

Flow

Kanban

Continuous software development

ABSTRACT

Managing flow is fundamental to continuous development, particularly in knowledge intensive work activities such as software development. However, while numerous articles describe flow tools and practice there is little research on their application in context. This is a significant limitation given that software development is a highly complex and socially embedded activity. This research applies activity theory (AT) to examine the adoption of flow techniques by using the multiple-case method in two companies. AT is particularly pertinent in this study as it identifies contradictions, which manifest themselves as problems such as errors or a breakdown of communication in the organisation and congruencies between flow techniques and the development context and indeed contradictions between components of flow techniques themselves. Rather than view contradictions as a threat to flow or as an argument to abandon, a theoretical contribution of this study is that it shows how contradictions and congruencies can be used to reflect, learn, and identify new ways of structuring and enacting the flow activity. It also provides an immediate practical contribution by identifying a set of lessons drawn from the cases studied that may be applicable in future implementations of flow techniques.

© 2016 Elsevier Inc. All rights reserved.

1. Introduction

Flow is a fundamental tenet of contemporary lean thinking, and is seen as the primary method to transition from agile to true continuous software development (Anderson, 2013; Fitzgerald and Stol, 2015; Olsson and Bosch, 2014; Poppendieck, 2002; Reinertsen, 2009; Tichy et al., 2015). Flow is about managing a continuous stream of value creating activities throughout the entire development process (Anderson, 2010; Reinertsen, 2009; Petersen and Wohlin, 2011; Poppendieck, 2002). This approach differs from traditional project management through its emphasis on managing queues rather than project phases and milestones (Power and Conboy, 2015; Anderson, 2013; Anderson et al., 2011; Ikonen et al., 2011). Flow techniques have been well received by those in software development, and there is evidence to suggest that awareness and indeed use of these methods is becoming quite prevalent in software development practice (Anderson, 2013; Nord et al., 2012; Petersen and Wohlin, 2011; Poppendieck and Cusumano, 2012; Power and Conboy, 2015; Reinertsen, 2009). For example, the flow concept is useful in considering continuous software engineering which emphasizes a continuous movement, rather than focusing on agile methods *per se*

(Fitzgerald and Stol, 2015; Fitzgerald and Stol, 2014). Flow encourages collaboration between teams, measurement of value, costs, and technical metrics, and knowledge sharing, which are key characteristics of DevOps (Bang et al., 2013).

Initial claims that evidence to support flow techniques remain largely anecdotal in the software development field (Ebert et al., 2012; Sjøberg et al., 2012; Ikonen et al., 2011) are now being addressed. A systematic review by Ahmad et al. (2013) showed nineteen relevant studies to 2011. There are other relevant studies before and since (Nord et al., 2012; e.g. Petersen and Wohlin, 2011; Petersen et al., 2014; Sjøberg et al., 2012; Staron and Meding, 2011; Khurum et al., 2014) where Kanban adoption and usage (Cocco et al., 2011; Shinkle, 2009; Senapathi et al., 2011), the effect of bottleneck detection (Rutherford et al., 2010; Petersen et al., 2014) and visualisation of development on metrics such as velocity (Anderson et al., 2011; Power and Conboy, 2015; Polk, 2011) are examined.

However, the existing body of knowledge is limited by the fact that most studies only focus on specific elements of flow e.g. Kanban boards, bottlenecks or Cumulative Flow Diagrams (CFDs) or tend to focus on the textbook ‘vanilla’ version of these tools. While they demonstrate the effect of flow they usually do not examine the tensions and contradictions that often arise when multiple elements of these techniques are implemented as part of an overall flow programme. Activities are not isolated elements; they are influenced by other activities and other changes in their environment. For example, Kanban by itself does not guarantee success as

* Corresponding author.

E-mail address: denis.dennehy@nuigalway.ie (D. Dennehy).

it is a relatively basic control tool that needs to be supported by additional practices (Ikonen et al., 2011).

The depth and complexity of this interplay between activities is exacerbated further given the socially embedded contextual nature of software development. It is well accepted that culture, team dynamics, the development team's 'softer' skills and the development and solution context are critical determinants of software development success and that a method, practice or tool cannot be studied in isolation (Conboy, 2009; Ebert et al., 2012; Fitzgerald et al., 2002; Kitchenham et al., 2002; Lytinen and Rose, 2006; Petersen and Wohlin, 2009). These issues indicate a need, not just to study flow techniques, but also to study them in the environment within which they must be implemented in and cognisant of in order to be effective.

This research draws on AT as it provides a theoretical lens specifically designed to study (i) multiple activities and practices in a multilevel, stratified manner, and (ii) to identify the contradictions and congruencies that emerge when people interact and use artifacts (e.g. Kanban board, cumulative flow diagrams) in a social context (Kaptelinin, 1996; Karanasios and Allen, 2014; Mursu et al., 2007; Spasser, 2002).

AT has inspired a number of theoretical reflections on software development (Allen et al., 2013; Bertelsen and Bødker, 2000; Chen et al., 2013; Hasan, 1998; Korpela et al., 2001; Kuutti and Molin-Juustila, 1998; Vermeulen et al., 2016). Ultimately, contradictions and congruencies interrupt or enable the fluent flow of work (Helle, 2000) and so AT is therefore particularly pertinent in the context of this study. Contradictions "indicate a misfit within elements, between them, between different activities or different development phases of a same activity" (Kuutti, 1995, p. 28). The resolution of contradictions within and between activities acts as a driver of change and may result in several levels of congruency which shape the success, normalisation and agreement between the use of the tool and the work activity (Hasan et al., 2010; Karanasios and Allen, 2014). This study examines contradictions and congruencies between flow techniques and the development context or indeed contradictions between the flow techniques themselves.

When we see how AT is constructively used in other fields (Foot, 2001; Helle, 2000; Kuutti, 1995), rather than view contradictions as a threat to flow techniques or as an argument to abandon them, contradictions can be viewed as an opportunity to reflect, to learn, and to identify new ways of structuring and enacting the flow activity. The lens of contradictions and congruencies within AT enable us to generate two interlinked research questions in the context of flow and its use in practice, namely:

- (1) *What is the motivation for adopting flow techniques?*
- (2) *What are the major contradictions in the implementation and use of flow techniques?*
- (3) *What are the major congruencies in the implementation and use of flow techniques?*

The next sections of the paper summarize the pertinent flow and AT literature and describe the theoretical basis and research approach adopted in this study. The findings from two 'revelatory' case studies are then presented and discussed, and through further analysis, the emerging set of (i) contradictions and (ii) congruencies in the implementation and use of flow techniques are presented. The paper concludes with a discussion of the implications and limitations of the study and possible avenues for future research.

1.1. Background to the concept of lean

Rooted in lean manufacturing, leanness has largely focused on cost reduction (Ohno, 1988), "the elimination of waste"

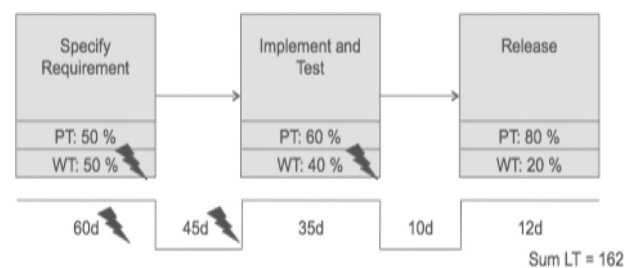


Fig. 1. Value stream mapping (source: Khurum et al., 2014, p. 6).

(Naylor et al., 1999; Ohno, 1988; Womack et al., 1990), and "doing more with less" (Towill and Christopher, 2002). However, the concept of lean has evolved over time with subsequent emphasis on the value and more recently the flow of work. Lean strives to deliver maximum value to the customer by reducing waste, controlling variability, maximizing the flow of information, focusing on the whole process, and not on local improvements (Anderson et al., 2011; Poppendieck, 2002). Lean is a mind-set, a mental model of how the world works (Poppendieck and Poppendieck, 2013). Lean thinking is guided by five interlinked concepts: value; value stream, flow, pull, and perfection (Wang et al., 2012).

1. Value: It is defined by the customer and it is paramount to have a clear understanding of what that is.
2. Value stream: A map that identifies every step in the process and categorises each step in terms of the value it adds
3. Flow: It is important that the production process flows continuously.
4. Pull: Customer orders pull product, ensuring nothing is built before it is needed.
5. Perfection: Striving for perfection in the process by continuously identifying and removing waste.

1.2. Principles of lean software development

Lean software development is a relatively new addition to the agile method family but is increasingly being adopted by software teams (Anderson et al., 2011). Poppendieck and Poppendieck (2003) published the first book that adopted lean principles from manufacturing and applied them to software development was adapted from lean production which consists of seven principles: 1) eliminate waste, 2) amplify learning, 3) decide as late as possible, 4) deliver as fast as possible, 5) empower the team, 6) build integrity, and 7) see the whole. These principles were later refined and are listed in Table 1 below.

Flow in lean product development is defined as "the progressive achievement of tasks along the value stream so that a product proceeds from design to launch, order to delivery, and raw materials into the hands of the customer with no stoppages, scrap, or backflows" (Womack and Jones, 2010, p. 306). Managing a continuous and smooth flow that delivers value to the customer is a key aspect of lean principles (Anderson, 2010; Petersen and Wohlin, 2011; Reinertsen, 2009). This results in production not being based on forecasts as commitment is delayed until demand is present to indicate what the customer really wants (Poppendieck, 2002).

1.3. Related work

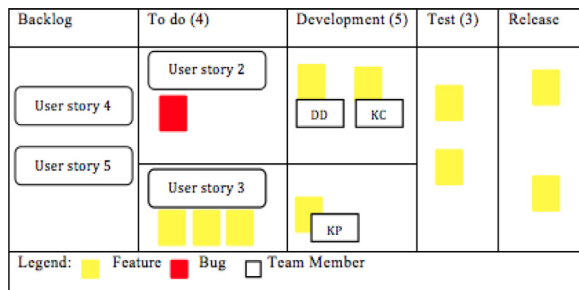
There are five commonly known techniques that are used to identify flow bottlenecks: (i) value stream maps, (ii) Kanban board, (iii) cumulative flow diagrams (CFDs), (iv) burn-down charts, and (v) line of balance status charts (Petersen et al., 2014).

Value stream maps: Value stream maps (see Fig. 1) are used to follow a specified item of work through the process in order to establish value added in each processing step (Petersen et al., 2014).

Table 1

Principles of lean product development.

Lean software development (Poppendieck and Poppendieck, http://www.poppendieck.com/)	The principles of product development flow (Reinertsen, 2009)	Kanban principles (Anderson, 2010)
• Optimize the whole • Focus on customers • Energize workers • Eliminate waste • Enhance learning • Increase flow • Build quality in • Keep getting better	• Use economically based decision-making • Understand behaviour of queues • Exploit variability • Reduce batch size • Apply WIP (work in progress) constraints • Use cadence, synchronisation and flow control • Use fast feedback loops • Decentralise control	• Limit WIP • Visualize workflow • Measure and optimize flow • Make process policies explicit • Manage quantitatively

**Fig. 2.** A Kanban board for software development (adapted from Nord et al. (2012)).

Value stream maps have been successfully applied to the context of software development to reduce lead-time (Mujtaba et al., 2010) and identifying value from four major perspectives: the financial, the customer, the internal business process, and the innovation and learning (Khurum et al., 2014).

A limitation of value stream mapping is that it only takes into consideration an individual work item in a single run through the process and it looks at bottlenecks retrospectively (Petersen et al., 2014).

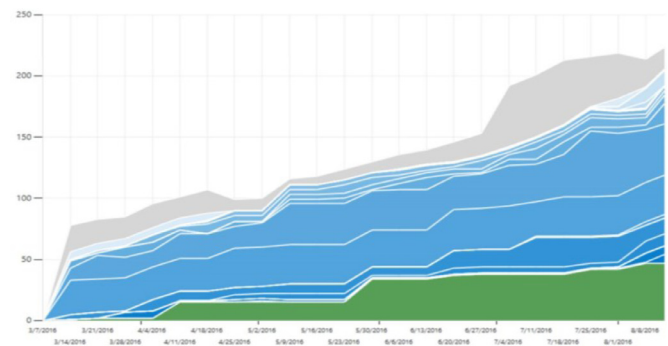
Kanban board: Kanban boards (see Fig. 2) are generally project focused and encourages the visualisation of workflow states as work passes moves through the organisation (Anderson, 2010; Power and Conboy, 2015; Petersen et al., 2014) by using a card system. These cards enable team members to observe work-in-progress and for the teams to self-organize by assigning their own tasks and to move work from a backlog to complete without direction from a project or line manager (Anderson, 2010). Explicit work-in-progress (WIP) limits are used to manage the quantity of work-in-progress at any given stage in the workflow, as well as explicit policies frequently called entry and exit criteria which determine when a work item can be pulled from one state to another (Power, 2014).

A limitation of using physical Kanban boards to manage multiple, complex projects is that it can lead to losing the overview (Petersen et al., 2014).

Cumulative Flow Diagrams (CFDs): CFDs are rooted in queuing theory and is another tool that is used to visualise and manage workflow states. CFDs show the amount of work in each of the defined work states, which can be viewed as queues (Power, 2014). CFDs are useful for understanding the behaviour of queues, and for diagnosing problems (Power, 2014; Reinertsen, 2009). An example of the construction of a CFD is shown in Fig. 3. The x-axis shows the time-line and the y-axis shows the cumulative number of items.

CFDs show an aggregated view of work in the software development cycle and are backward looking (Petersen et al., 2014).

Burn-down charts: Burn-down charts are used in agile methodologies, such as Scrum to show an aggregated view of the amount of work that remains in a current iteration, typically a two week

**Fig. 3.** Cumulative flow diagram (source: project partner).

sprint (Kniberg and Skarin, 2010; Petersen et al., 2014). An example of a burn-down chart is shown in Fig. 4. The y-axis shows the number of work items remaining to be completed after a specific time (x-axis).

A limitation of burn-downs chart is that they are backward looking and it is difficult to identify if a specific work item is soon to become a bottleneck (Petersen et al., 2014). In addition, because it treats work as a fixed commodity rather than an expandable, it limits their use when change occurs during the tracking period (Erdogmus, 2010).

Line of balance status chart: The line of balance status chart (see Fig. 5) is a linear technique that allows the balancing of operations such that each activity is continuously and effectively performed (Arditi et al., 2002, 2001). The line of balance status chart is a backward looking and presents an aggregated view (Petersen et al., 2014).

This linear technique is best suited to projects involving activities of a repetitive nature (Al Sarraj, 1990).

Although these five techniques: (i) value stream maps, (ii) Kanban board, (iii) cumulative flow diagrams (CFDs), (iv) burn-down charts, and (v) line of balance status charts provides opportunities to identify bottlenecks, none of them provide a systematic way to integrate different viewpoints (Petersen et al., 2014).

Complementary to the visualisation tools that are used to identify bottlenecks in flow are a number of key metrics used for understanding and managing the flow of work in software development practices (see Table 2). Reinertsen (2009) posits that since flow-based product development identifies different causes for success, it follows that it emphasises different metrics.

From the perspective of AT, the techniques listed previously are cultural-historical tools that act as types of mediator. Generally, software teams that are more advanced in the use of flow techniques would use a combination of these techniques. However, in the context of this study, we focus mainly on the Kanban board as the mediating tool as this was the primary tool used by the software teams and to a lesser degree CFDs. This point is discussed further in the next section.

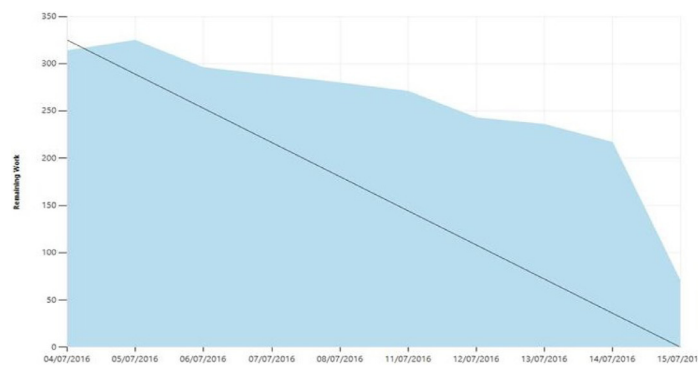


Fig. 4. Burn-down chart (source: project partner).

Table 2

Metrics used to manage flow (Power and Conboy, 2015; Reinertsen, 2009).

Metrics	Description
Queue size	Queues are the underlying cause of various forms of economic waste: longer cycle time, increase risk, increase overhead, and lower quality.
Throughput rate	Reveals the rate of work through the system over time and when combined with demand analysis shows how much work is value demand (e.g. Customer requesting something new such as a new product feature) versus failure demand (e.g. When a product or product feature does not meet the customer's needs and generates additional work).
Cycle time	Shows how long individual work items spend in specific workflow states and are used to understand how work flows through individual or combinations of work states.
Lead time	Shows how long it takes for individual work items to move through a system – from initial request to delivery to end-user.

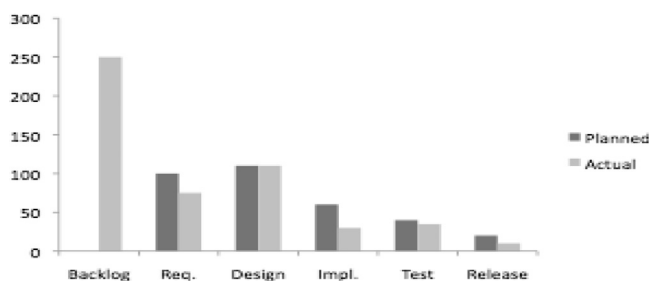


Fig. 5. Line of balance chart: (source: Khurum et al., 2014, p. 6).

2. Contributions of activity theory

Contemporary thinking on AT, known as third-generation AT (cf. Engeström, 1987) is rooted in the lineage of cultural-historical psychology and specifically concepts of ‘mediation’ (Vygotsky, 1978), Ilyenkov’s (1974) notion of ‘ideal forms’ (signs, symbols, language) and the triangular model of ‘a complex, mediated act’—hence why the notion of mediation via artefacts is central to AT (Kaptelinin, 1996). AT is broadly defined as a “philosophical and cross-disciplinary framework for studying different forms of human practice as historically developing cultural systems” (Kuutti and Molin-Juustila, 1998). The strengths of AT are the interaction in a social context, and the notion of dynamics and development of a work activity (Mursu et al., 2007). AT serves as a lens to gain insight into tool-mediated, goal-directed human activity in context (Cole and Engeström, 1993; Ditsa, 2003).

AT is considered to be rooted in practice (Schatzki, 1998) as it focuses on the relationships between material action, mind and society by exploring the links between thought, behaviour, individual actions and collective practices (White et al., 2016). Practices constrain and enable the process through which an activity occurs, they influence how recognition of an action is given, and what types of relationships may be formed in an activity context (White et al., 2016; Packer and Goicoechea, 2000). Mursu et al. (2007) consider AT to be a prospective approach to work-oriented and participatory software development. Certainly, understanding of the rich, emergent interactions between technology and its so-

cial context (people, culture, organisation, management) is among the best contributions that academia can offer practice (Lee, 2001).

2.1. Components of activity theory

The basic premise of AT is that a subject (a person or collective) is driven by a motivation(s) to act upon an object (a person, collective, or thing) using cultural-historical tools which include technologies, mental tools, language (Karanasios and Allen, 2014). The term ‘tool’ is emphasised in AT and includes meaningful items (signs, symbols and languages) that act as types of mediator (Allen et al., 2013). Kuutti (1995) highlights that although cultural signs such as words and tools mediate human activity, the tool can be enabling and limiting at the same time.

The unit of analysis in AT is an activity which links individual actions to context. Actions without context are meaningless (Mursu et al., 2007). In this sense activity is ‘purposeful’ rather than ‘busy-ness’ (Hasan et al., 2010).

The elements of AT (listed in Table 3) are used in the context of this study as a lens to interpret the actions of individuals who are members of a software development community and whose work is mediated by the flow tools (i.e. Kanban boards) mentioned previously.

Acknowledging the context in which an individual carries out an activity Engeström (1987) highlights the influential nature and interrelatedness of the larger social context by adding rules and norms, community and division of labour. Engeström’s activity model is illustrated in Fig. 6 below.

The object is represented by an oval to indicate that object-oriented actions “are always, explicitly or implicitly, characterised by ambiguity, surprise, interpretation, sense making, and potential for change” (Engeström, 2005, p. 61). Identifying and understanding contradictions and congruencies within the activity system can lead to change within the activity system. Contradictions and congruencies are discussed in the next section.

2.2. Contradictions and congruencies in an activity system

Contradictions are viewed in AT as “the motor of change” and refer “to anything within the system that opposes the overall mo-

Table 3

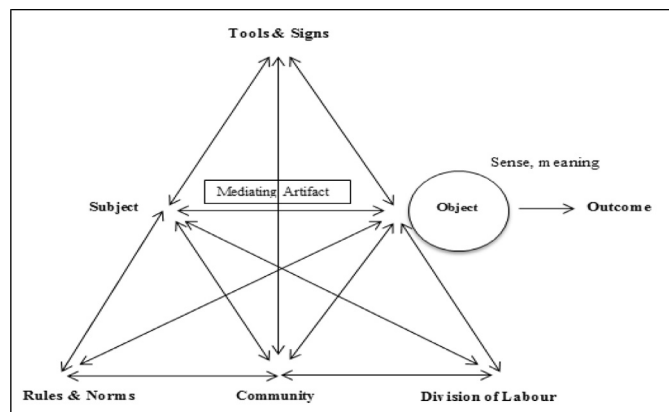
Elements of AT (adapted from Engeström, 1993, pp. 67–68).

Elements of AT	Description
Subject	Refers to the individual or subgroup whose agency is chosen as the point of view in the analysis.
Object	Refers to the raw material or problem space, which is moulded or transformed into outcomes with the aid of physical or symbolic, external and internal tools (mediating instruments and signs). An object can be a material thing, but it can also be less tangible (a plan) or totally intangible (a common idea) as long as the participants of the activity can share it for manipulation and transformation.
Community	Comprises of multiple individuals and /or sub-groups who share the same general object
Division of labour	Refers to both the horizontal division of tasks between members of community and the vertical division of power and status.
Rules	There is a set of rules covering what it is to be a member of this community. It refers to the explicit and implicit regulations, norms and conventions that constrain actions and interactions within the activity system.
Tools	In each step of the transformation process a different set of tools and instruments is used. The collection of these tools has a history: it is a result of a process of accumulation and rejection at both company and team level and additions and deletions to it may occur during any project.
Outcome	Is achieved when an activity is undertaken by a subject that is motivated to act on an object (Allen et al., 2013) which is eventually transformed by the activity system. Outcomes are affected by the actions of the activity system (Nardi, 1996), which can be undesirable (c.f. Allen et al., 2014) or unintended, and can potentially change the process of the activity system (c.f. Canary and McPhee, 2009).

Table 4

Levels of contradictions in activity systems.

Contradiction level	Engeström's (1987) definition
Level 1: Primary contradiction	When activity participants encounter more than one value system attached to an element within an activity that brings about conflict.
Level 2: Secondary contradiction	When activity participants encounter a new element of an activity, and the process for assimilating the new element into the activity brings about conflict.
Level 3: Tertiary contradiction	When activity participants face conflicting situations by adopting what is believed to be a newly advanced method for achieving the object.
Level 4: Quaternary contradiction	When activity participants encounter changes to an activity that result in creating conflicts with adjacent activities.

**Fig. 6.** Engeström's activity model (1987, p. 78).

tive of the system, the aim or purpose that subjects within the system are individually or collectively striving toward" (Allen et al., 2013, p. 840). Most often real activities are continuously in the process of working through some form of contradiction which can occur within and between activity systems (Kuutti, 1995; Murphy and Manzanares, 2008). Within an activity system contradictions are a cultural-historical force that lead to activities that are continuously evolving and transforming (Allen et al., 2013) in which "equilibrium is an exception and tensions, disturbances and local innovations are the rule of thumb and the engine of change" (Cole and Engeström, 1993, p. 8). Contradictions are defined as "a misfit within elements, between them, between different activities, or between different developmental phases of a single activity" (Kuutti, 1995, p. 12).

Engeström (1987) proposes four levels of inner contradictions in activity systems (see Table 4). However, due to the early stage of adopting flow practices and inevitably the early stage of this study, we focus on primary and secondary contradictions.

Contradictions may not be obvious, openly discussed, visible or discussable, as well as be culturally or politically challenging to confront (Allen et al., 2013; Capper and Williams, 2004). A related problem is that contradictions are not always acknowledged or resolved (Murphy and Manzanares, 2008; Nelson, 2002). However, contradictions can be resolved through tool-mediated change which may result in several levels of congruency that ultimately have a positive influence on the activity system (Karanasios and Allen, 2014). These congruencies can be immediate where things or activities work better in some way or potential areas for longer term congruencies (Allen et al., 2014). As an activity system is continuously working through contradictions, the activity system is a virtual disturbance and innovation-producing machine (Ditsa, 2003).

3. Research method

This study uses a multiple-case design to allow a cross-case analysis to develop more sophisticated descriptions and powerful explanations (Miles and Huberman, 1994). Leveraging the case study is valuable for "problems... in which research and theory are at their early, formative stages and sticky, practiced-based problems where the experiences of the actors are important and the context of action is critical" (Benbasat et al., 1987, p. 369).

The rationale for adopting a case study strategy is to "capture the circumstances and conditions of an everyday or common place situation" (Yin, 2009, p. 48). Therefore, to fulfil the objective of this research, studying a case focuses on 'contemporary events' (Yin, 2009) and facilitates engagement with those 'living the case' (Stake, 2000) to unearth the complexities of optimising flow in agile teams.

The six most commonly used sources of evidence used in case study research are: (i) documentation, (ii) archival records, (iii) interviews, (iv) direct observations, (v) participant-observation, and physical artifacts (Yin, 2009). In this study, interviews, direct observations, participant-observations, and physical artifacts were used as sources of evidence. The advantage of using multiple sources of

Table 5
Background to cases.

Element	Company A	Company B
Domain	Technology solutions services and support.	Telecommunication software and infrastructure
Market	Bespoke development	Market-driven development
Product maturity	Less than 5 years	Older than 3 years
Product customisation	Customisable	Customisable
Organisational structure	Divisional	Matrix
Distribution of organisational units	Distributed (internationally)	Distributed (internationally)
Global workforce	>100,000	>115,000
Number of software development teams locally/members	20/160	40/360
Software development team location	Distributed	Distributed
Software Methodology	Scrum	Scrum
Years using Scrum	3	8
Software development practices	Time boxed two-week sprints; interaction (daily stand-up meetings using video conference software)	Time boxed two-week sprints; interaction (face-to-face daily stand-up meetings).
Use of visualisation techniques	Physical Kanban board, burn-down charts, CFDs	Physical Kanban board, burn-down charts, CFDs
Software development tools	Team Foundation Server	JIRA

evidence is the development of ‘converging lines of inquiry’, a process of triangulation and corroboration as this increases the precision of the empirical research, thus making the findings and conclusion more convincing (Yin, 2009; Runeson and Höst, 2009).

In line with guidelines for conducting and reporting case study research in software engineering proposed by Runeson and Höst (2009), the key characteristics of this case study are: 1) it is of a flexible type, to cope with the complex and dynamic nature real world phenomena such as software engineering, 2) it uses qualitative data from multiple sources of evidence in a planned and consistent manner to derive conclusions from a clear chain of evidence, and 3) it adds to existing knowledge by being based on an established theory (e.g. AT).

3.1. Background to the cases

This study involves two multinational companies located in Ireland that were piloting flow techniques with two local software development teams in each company (Table 5). The aim of the pilot projects was to understand how flow techniques could be applied to the context of software development, as well as to identify contradictions (problems) and congruencies (solutions) before rolling out an organisation wide flow initiative. Due to the early stage of adopting flow practices both companies had not yet adopted the full suite of flow techniques (value stream maps, Kanban board, CFDs, burn-down charts, and line of balance status charts) into their daily practices. As a result, this study primarily focuses on the use of the physical Kanban.

Due to the sensitive nature of internal work processes and to protect the identity of the companies, pseudonyms are used, Company A and Company B. The cases were purposefully selected because (i) both companies were project partners on a funded project that focused on the application of flow techniques in software development practices, (ii) both companies were in the very early stage of adopting flow techniques in the teams studied (<six months), (iii) ease of direct access to interviewees involved in the flow project, and (iv) access to both companies permits the research team to observe the use of flow techniques in real-time.

Due to the early stage of adopting flow practices both companies had not yet adopted the full suite of flow visualization techniques (value stream maps, Kanban board, CFDs, burn-down charts, and line of balance status charts) into their daily practices. As a result, this study primarily focuses on the use of the physical Kanban as all interviewees were familiar with it and to a lesser degree CFDs as not all interviewees had used it but could talk about it as a concept.

Interviewees were selected using role-based sampling (Ikonen et al., 2010) so that they represented the population being stud-

ied and are listed in Table 6. As these job titles are reflective of the culture and structure of each company, their meaning may not translate universally. Therefore, a brief explanation of each role is included in Appendix A.

As noted, the participants varied in terms of their role, previous experience and use of software methods. In total, fifteen interviews were conducted, one interview per job title listed.

3.2. Data collection and analysis

To improve reliability and repeatability, an ‘audit trail’ was created from data collection through to drawing of conclusions. Data were collected over a three-month period in 2015, primarily through fifteen; purposefully selected, face-to-face interviews. This technique is well suited to case study data collection, and particularly for exploratory research such as this as it allows expansive discussion to illuminate factors of importance (Oppenheim, 1992; Yin, 2003). Interviews lasted between 40 and 90 min. Although the process of data collection and analysis have been rigorous, this was an exploratory study and therefore only weak conclusions can be drawn (Kitchenham et al., 2002).

An interview protocol was prepared based on the components of AT (Fig. 2) and specifically its six underlying constructs (*tools and signs, subject, object, rules and norms, community and division of labour*). These provided a list of ‘intellectual bins’ (Miles and Huberman, 1984) to structure the data collection and for thematic analysis. A sample of the data analysis is shown in Appendix B. Questions were largely open-ended, allowing respondents freedom to convey their experiences and views, and expression of the socially complex contexts that underpin software development generally and the research questions in this study specifically. The interviews were conducted in a responsive, reflexive manner (Rubin and Rubin, 2005; Wengraf, 2001), allowing the researchers to follow up on insights uncovered mid-interview, and adjust the content and schedule of the interview accordingly. All interviews were recorded, transcribed, proof-read and annotated. In any cases of ambiguity, clarification was sought from the corresponding interviewee, either via telephone or e-mail. Interviews were supplemented by 20 hours (12 hours in company A, and 8 hours in Company B) of direct observations and participant-observations using the Kanban boards in practice.

3.3. Threats to validity

In order to evaluate the validity of this study, the authors have used the validity framework presented by Wohlin et al. (2012):

Construct validity relates to obtaining the right measures for the concept being studied as some threats relate to the design of

Table 6
Interviewee profiles.

Company A	Years of experience	Company B	Years of experience
(1) IT director	25	(1) Network program flow manager	25
(1) Portfolio manager	30	(1) Network program flow & visualisation manager	20
(1) Systems integration manager	20	(1) Network integration manager	20
(1) IT delivery manager	16	(1) Network program manager	18
(1) Technical program manager	15	(1) Network program area manager	22
(1) Financial controller	21	(1) Product development unit manager	15
(1) Senior business analyst	12	(1) Product development department manager	24
(1) Software development consultant	18		

the experiment and social factors (Wohlin et al., 2012; Runeson and Höst, 2009). The main threat in this context was that the researcher influences the outcome of the study with his or her presence. This threat was reduced as the researcher used data triangulation which essentially provided multiple measures of the same phenomenon (c.f. Yin, 2009). Training on the concepts and terminology of Kanban and flow was conducted to ensure that interviewees when responding were familiar with these techniques.

Conclusion validity relates to the ability to draw the correct conclusion about the relationship between theory and observation. While this risk cannot be eliminated, it was reduced by taking following actions: (i) two researchers were involved in the interpretation of the transcribed interviews and notes, (ii) as stated earlier, a full 'audit trail' from data to conclusion was maintained, (iii) an independent expert software method researcher reviewed the initial interpretation of the data, and (iv) and the findings were reviewed by a senior flow practitioner at one of the companies. Also, as evident from Table 6, the interviewees had very diverse roles and experience. As with any software case research, such experiences will have a bearing on each participant's responses. To ensure validity, the interviewer allowed participants to convey their opinion but ensured that each opinion had some evidence drawn from the project being studied, as distinct from evidence from some other experience or indeed no evidence at all.

External validity relates to the extent to which the study results are generalisable (Wohlin et al., 2012; Runeson and Höst, 2009; Kitchenham et al., 2002). Generalisability is a common issue in case studies where there is usually a rich and complex context. In order to know what degree the results of a study can be generalised, it is extremely important to describe the research context (Petersen and Wohlin, 2009; Kitchenham et al., 2002). Following the checklist for empirical studies proposed by Petersen and Wohlin (2009), the context elements for this study are listed in Table 5 which describe the most relevant subset of elements for the conclusions drawn in relation to the objective of this study. Future replication in different contexts such as regulated environments is necessary to further strengthen generalisability.

Internal validity relates to causal relationships and ensuring that it is not a result of a factor that was not measured or the researcher had no control over, or other unexpected sources of bias (Wohlin et al., 2012; Runeson and Höst, 2009; Kitchenham et al., 2002). The aim of the study was not to establish a statistical causal relationship. One relevant threat however is whether improved performance can be attributed to the observed flow implementation in the cases. Given the scale of the case companies, the number and complexity of parallel activities, the dynamism of the cases and their surrounding environment, and the early stage of flow adoption, it is not possible to establish such a causal link.

4. Results and analysis

In this section, we use an AT analysis to examine two key interrelated activities: (i) the motivation for adopting flow tools and (ii) the flow tool-mediated activity. Although there is significant

overlap between these phases, we use the lens of contradictions and congruencies to move between them to illustrate how the notion of providing flow techniques to software managers and their teams emerged from cultural-historical contradictions and came to present several levels of congruency.

The AT constructs (*tools and signs, subject, object, rules and norms, community and division of labour*) are used to structure and present the results and for the purpose of thematic analysis. Fig. 7 illustrates a flow tool-mediated activity system that was derived from a synthesis of the data collected.

Fig. 7 illustrates a flow tool-mediated activity system. In this instance, we focus on the decision making activity of a project manager (the subject) who decides to add a new user requirement in the form of a new feature/functionality (shared object) which is mediated by flow techniques in order to satisfy the motivation of maximising efficient use of resources. This interaction is mediated by physical tools (Kanban, CFDs) and mental tools (expertise), which mediate the subject's thoughts and behaviour (c.f. White et al., 2016). The activity involved a broader community that includes software developers, testers, and analysts. The inclusion of *subject, community* and a *shared object* within one unit of analysis encapsulates the multiple voices that frame an activity. The activity is also governed by: division of labour (i.e. new responsibilities and decision-making as a result of adopting flow techniques); rules and norms (i.e. management style, communication, and company policies), and the software methodology used (e.g. Scrum). Both companies were considered by the interviewees to be mature in terms of using an agile software methodology, which manifested a culture of self-forming and self-organising teams. The broken lines labelled (A, B1, B2, C1, C2, D1, D2) in Fig. 7 represent the sources of contradictions identified within the activity system and are discussed in the next section.

4.1. Primary contradiction and emergent congruency

Using AT to understand the motivation for adopting flow techniques (Research Question 1) revealed one primary contradiction (see line A, Fig. 7), namely that the managers' 'shared object' for participating in the adoption of flow techniques was not aligned with the software teams' 'shared object'.

The initial motivation underpinning the introduction of flow techniques in Company A was traced back to approximately twelve months prior to the initial pilot project when the IT Director recognised a need for a fluent flow of work, but acknowledged that the current software methodology in use did not provide this. While in Company B the initial motivation underpinning the introduction of flow techniques was traced back to approximately eighteen months prior to the pilot project when the Network Program Flow Manager and the Network Program Flow & Visualisation Manager recognised the need to address significant delays in the deployment of new features for the new network management system, a core product of the company.

Transparency of work was a major contradiction that existed within the activity norms. The current dominant work norms sup-

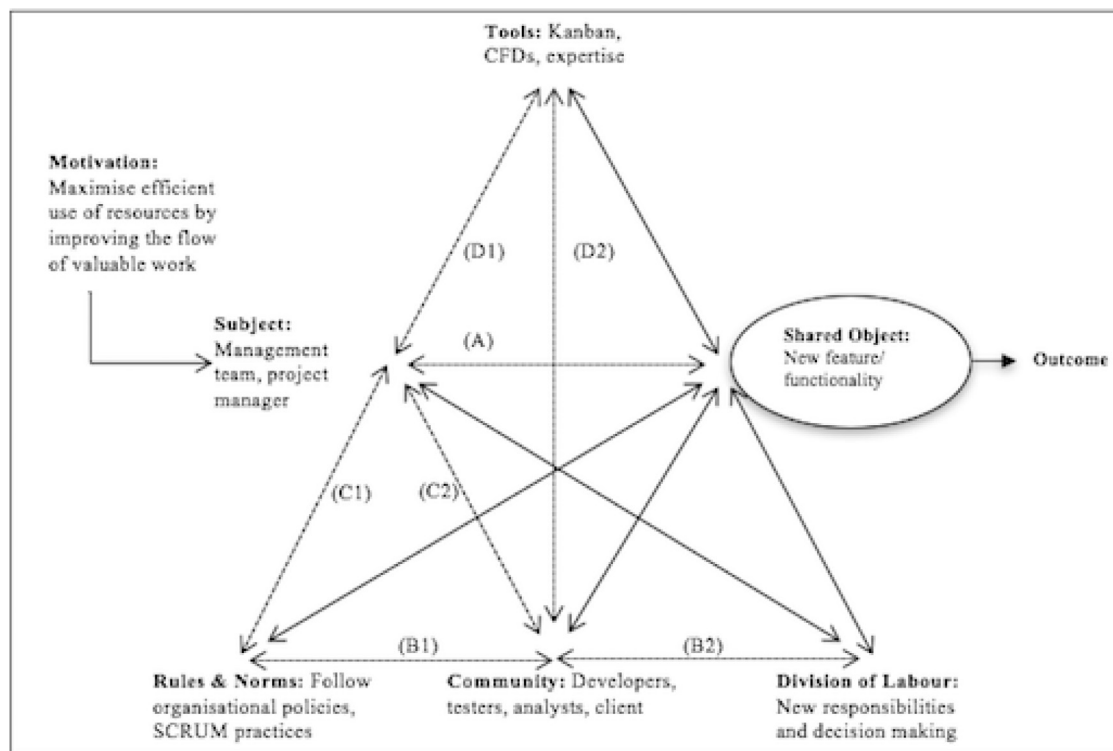


Fig. 7. An AT representation of a flow tool-mediated activity system.

ported behaviours that were drawn from earlier activity systems predicated on self-organising and self-managing teams, who decided what work they would commit too and how long it would take. In contrast, the adoption of flow tools provided an alternate set of rules and norms predicated on the need for teams to make their work visible and open to scrutiny by managers and their peers. A manager from Company A explained that the motivation to adopt flow practices was

to provide greater visibility into what teams are doing... it will definitely improve the feedback loops [and] it will expose resourcing constraints and even capacity utilisation.

A manager from Company B explained that flow was viewed internally “as a way of promoting performance improvement”. However, he acknowledged that people will “need to be disciplined, it’s going to hurt in the beginning, and after that I think they will have a better feeling of accomplishment”. As this contradiction between the existing and new norms emerged over time this primary contradiction became a source of ambiguity and uncertainty about the acceptability of use of flow techniques in software development practices.

Emergent congruency to primary contradiction

This contradiction was resolved with the norms of making work visible a dominant feature in the daily work activity of the software teams. The emerging congruency between the new norms, the non-material tools (language), and the physical Kanban tool provided managers of the pilot projects with a rationale and legitimisation for their flow initiative.

Although this misalignment of objects may seem obvious, by examining the interactions between managers, the software teams, and flow techniques, AT provides insight into the subsequent secondary contradictions and emergent congruencies, which are discussed in the next section.

4.2. Secondary contradictions and associated emergent congruencies

Within the context of the flow tool-mediated activity system (Fig. 7), three secondary contradictions and emergent congruencies were identified. These are discussed in the remainder of this section as it provides answers to research question two (*What are the major contradictions in the implementation and use of flow techniques?*) and to research question three (*What are the major contradictions in the implementation and use of flow techniques?*). Taking into consideration the sensitivity of contradictions and congruencies and to protect the identity of interviewees, no identifying codes have been assigned to quotes used in this section.

Secondary contradiction 1: change in organisational culture
Prior to the implementation of flow (a physical Kanban), software teams were relatively free to decide what new pieces of work (stories) they would take on and how long this piece of work would take to complete. The anticipated and subsequent use of a physical tool to inform decision making, and increase visibility and accountability created a contradiction. Effectively, this contradiction (lines labelled B1 and B2 in Fig. 7) manifested in both changes to the division of labour and, in part as a result of this, in changes to the rules and norms which controlled decision making and behaviour within the activity system. In this context of organisational culture (rules and norms) a manager at Company B believed that “probably the biggest impediment to flow is getting the organisation to embrace it, to embrace change. I think we have to be softly, softly as we bring that in”.

A deeper analysis revealed that this change could be linked to the creation of another contradiction between the tools that was caused by the introduction of flow. That is, between the expertise and knowledge of the project manager (mental tool) and the capability offered by the Kanban (physical tool). This led to a shift in rules and norms concerning the activity. Evidence of this contradiction is captured by a manager at Company B who acknowledged that he was initially sceptical of the tool (Kanban) because it “looked very gimmicky and I just don’t trust pieces of paper stuck to the wall”. Another manager at Company B commented that

I think there is a fundamental belief that it will improve productivity but how ever can you measure productivity in software development. I just don’t believe it is possible. I think

organisations can put too much effort into measuring stuff instead of putting that energy into doing stuff.

However, the same manager conceded that having seen the physical Kanban in operation for a while he became aware of how people were getting benefit from it, such as identifying a build of work in certain state (development) which can be discussed during the daily stand ups. Such views reveal how a contradiction between a subject (a manager) and a tool could have the potential to undermine the adoption of flow techniques or become a congruency.

Emergent congruency to contradiction 1:

After an initial period of using the physical Kanban, a congruency in the work activity emerged between the subject and the tool, as well as the community. A manager at Company B explained that by using the physical Kanban it was “enacting everyone involved by basically seeing where the problems are”. He elaborated on this point saying that on one occasion

We actually decided the engineers involved in the early phases of the development, that there was no point in them doing any more work because all the work was backed up further on in the project for development. A decision was made to release some of the engineers, solution analysts and bring them back into teams to help in coding and to remove that bottleneck.

Identifying early wins in the use of the flow tools assisted managers to secure the buy-in of subjects and members of the community who were apprehensive about adopting flow techniques and may even consider flow to be a software development fad driven by consultants.

Secondary contradiction 2: over-control versus the need to monitor

Managers from both companies were aware of the negative perception that the use of flow techniques may have on the community – over control versus the need to monitor. An interviewee at Company A believed that “it [Kanban] exposes people, not expose them in a bad way to say he is not doing his job”. Another interviewee at Company A expressed similar sentiment

There would definitely have been some cynics who would have said everybody needs to be working on something and we need to know what those things are and they need to work as hard as they possibly can. The idea of leaving guys idle waiting to solve problems would have been a huge cultural, mental challenge for some people in this organisation.

This contradiction was also evident in Company B as one manager stated, “we need to have control of what comes in and what comes out”. However, his colleague explained that “software teams are averse to people tracking them”. In this context, a secondary contradiction (lines labelled C1 and C2 in Fig. 7) was the dual nature of Kanban (tool) as a decision making tool, and as a tool for management to monitor the outputs of individuals and teams which challenged the current organisational norms.

Emergent congruency to contradiction 2:

A possible congruency would be how managers introduce and explain how flow techniques will enhance a team’s work activity, as explained by this manager at Company A “the message to developers has to be carefully given because any change rocks the boat”. A similar view was shared by his colleague who believed that “once they see that, not only from a management standpoint but from a team standpoint, that progress is being made and they can measure how well they are doing, or not, and what we need to do in order to improve”. In the context of using the physical Kanban for better engagement between management (subject) and the development teams (community), a manager at Company A believed that “if you can feel it and touch it, you believe it to be re-

al...then people will take on responsibility for that piece of work”. Building on this congruency, his colleague commented that “people always want to work smarter, they always want to have more time and if this gives them back time to do things in their working day rather than sprint, sprint, sprint and eventually burnout”. Understanding the temperament of software teams and communicating the need for monitoring can act as congruency to addressing the fear of over control.

Secondary contradiction 3: disruption to work practice

A number of managers initially interpreted the adoption of flow techniques as interrupting their established work activity pattern. This can create a contradiction between the subject and the tool (as well as between the tool and the community) as managers (and members of the community) learn to use and interpret flow tools. The lines that represent this contradiction are labelled D1 and D2 respectively in Fig. 7. A manager at Company B believed that “what is up on the board should be in JIRA because we’re trying to get the same data in two different places and that is a challenge... to me there is a huge amount of work in the physical board”. Along similar lines, another manager at Company B stated, “at the start people were saying it is in JIRA, why would we need an illustration on the wall? This is something we can’t maintain; it doesn’t make sense”. This contradiction was exasperated by the low attendance during an in-house training session on the use of CFDs, as highlighted by this manager, “we’ve set up an operations review on a monthly basis using this tool like the CFD and the control charts. The take up has been poor, we have only set up one activity so far but it’s all got to do with understanding how to use it”.

Emergent congruency to contradiction 3:

Contradiction 3 can be viewed as a temporal contradiction (c.f. Karanasios and Allen, 2014) which can be overcome as management (and members of the community) become more confident in the use of flow tools and flow becomes less prominent and smooth and people in general become more familiar with flow tools.

An AT representation of a flow tool-mediated activity and the identification of three contradictions and congruencies: division of labour and organisational culture; over control versus the need to monitor; and, disruption to work practice, provides a unique and much needed baseline for discussion in the next section.

5. Discussion, limitations & future research

A strength of AT is that it provides a lens to understand the context of flow in a particular setting and allows researchers to consider the cultural-historical influences, providing critical reflection on its emergence and use, rather than a narrow focus on the technical aspects of the methods themselves. We examined the ways in which contradictions and congruencies materialised in the emergence, implementation and use of flow techniques. In addressing the research questions, we showed that flow techniques were implemented in order to overcome the major contradiction in software development practice - to make work visible and open to scrutiny by managers (subjects) and other members of a software development team (community). To a lesser degree, congruencies in the use of flow techniques were embryonic as both companies were in the early phase of adopting flow. However, the argument is not that the number of congruencies exceeds the number of contradictions, but rather that congruencies show resolution of contradictions which is important in explaining the evolution in the activity and the tools used to mediate that activity. This is observed in the study, through the process of recognising contradictions, which can interrupt the fluent flow of work (Helle, 2000), potential resolution of the contradiction through the experience of intended and unintended congruencies.

Four key contradictions were identified in the activity theoretic analysis.

(i) The first contradiction was the misalignment of the shared object between management and members of the software team, which was mediated by the use of flow techniques. Although in this instance it was the use of a Kanban boards teams more advanced in the use of flow techniques would also use a range of flow techniques (CFDs, value stream maps, and control charts), which could intensify this misalignment. Integrating the Kanban board into the daily work practices provided visibility of work and during the daily stand-ups it shifted the focus from a people-centred meeting to a board-centred meeting, which acted as a congruence to demonstrate that utility of Kanban and to further legitimise the flow initiative.

Identifying a misalignment in the early stage of adopting flow techniques is important because it provides an opportunity to focus on understanding and addressing a conflict the lies with the motivations of management and teams rather than a basis to reject flow techniques. Previous research (Stray et al., 2011; Shinkle, 2009; Kinoshita, 2008; Nikitina and Kajko-Mattsson, 2011) has mostly focused on the assimilation of flow practices from a software team perspective. However, management teams also influence successful adoption of flow practices. To the best of our knowledge this study is the first to provide a management perspective on flow practices by using AT analysis.

(ii) The second contradiction manifested in both changes to the division of labour and to the rules and norms, which controlled decision-making and behaviour within the activity system. Resolution to the contradiction emerged through the use of the tool, which enacted both the subject and community to seeing where problems are in the software development process and collectively proposing solutions to the problem. Previous research (Birkeland, 2010; Greaves, 2011) has suggested that a cultural change over a period of time is required to realise the full benefits of flow practices, as well as reprioritisation and adherence to the agreed policies to make a flow process work. Although using AT analysis in this study revealed similar findings to previous studies, we emphasise the importance of knowing the capacity of the organisation to take on such change. This is important to insure that the adoption of flow practices such as Kanban is not superficial because it is only when flow practices are infused into the daily software development work practices that an organisation will realise the full benefits of flow practices.

(iii) The third contradiction was the early perceptions that management will use flow techniques to over control individuals and teams. An emergent congruence was the use of appropriate terminology to explain the tangible benefits of flow to the community (software teams). While this congruence has not yet been fully realised, due to the early phase of adoption of flow, it does highlight the importance of language and style of communication at pre-implementation and post-implementation of flow techniques. Previous research (Ikonen, 2010; Anderson et al., 2011) highlighted the importance of being aware of the impact that (poor) communication and leadership within the context of identifying waste in Kanban-based software projects.

(iv) The fourth contradiction was the interruption to work, as managers' were conscious about satisfying the high expectations of their peers and the community, while also learning to use and interpret the meaning of flow tools. This is considered a temporal contradiction, which can be overcome as management become more confident in the use of flow tools, and people in the organisation become more familiar with the use of flow tools. To the best of our knowledge, previous research has not made this factor explicit, which may indicate that there is an assumption that the adoption of flow practices does not initially interrupt the fluent flow of work. Although it may only be a temporal interruption, we contend that unless appropriate training and support is pro-

vided, it could lead to loss of momentum and the flow initiative prematurely abandoned.

By examining the two interrelated activities (i) the motivation for adopting flow tools and (ii) the flow tool-mediated activity, we see that the introduction of flow techniques can create contradictions, as well as how congruency formed and resolved contradictions. However, there are limitations to this research. First, as both companies are in the early adoption phase of flow techniques, our activity theoretic analysis was restricted to this early phase. Secondly, the cultural-historical emergence of flow techniques is inherently time bound. Therefore, other contradictions are likely to emerge as both companies mature in the use of flow techniques and the congruencies that we consider to be embryonic may indeed transform. Thirdly, related to the previous limitation is that a contradiction is not always openly discussed and may be culturally or politically challenging to confront which means they may always be resolved. These limitations do however, present interesting directions for future research and are important when considering flow in the context of software development.

The study of flow and the application of AT to flow and indeed software development generally, can provide rich avenues for future research. For example, it is notable that while activity theory identify and resolve contradictions and congruencies, flow techniques such as Kanban, bottlenecks and root cause analysis perform a similar duty, although using different terminology. Also the former is a highly theoretical concept while flow techniques are practical in origin and application. Therefore the combination of the two with a view to a more rich, nuanced lens that combines theory and practice could provide a very interesting avenue for future research. Also further explanatory studies could seek to establish the impact of the contradictions identified in this by conducting a longitudinal study to examine the assimilation of flow techniques from initiation stage to infusion stage (c.f. Gallivan, 2001) or as conducting a cross-case analysis with companies using a suite of flow techniques.

6. Conclusion

In this paper, we focused on the emergence, implementation and use of flow techniques in an inherently socially embedded activity of software development which is highly complex and contextual. We explicated upon two key sets of contributions. The methodological contribution lies in the utilisation of AT to understand the phenomena of flow. AT permitted us to identify contradictions and congruencies between flow techniques and software development practices, and the dialectical interaction between contradictions and congruencies which can lead to a stage of change. Although the adoption of flow techniques by both companies is in the early phase and the associated research in the early phase of discovery, it does provide a much needed baseline for flow research and it presents a research area and theoretical lens that warrants further research. Further empirical research will be conducted with both companies as they progress with the adoption of flow techniques. In doing so, the body of knowledge on flow in software development environments will be advanced, thus enabling the creation of a flow maturity model. In addition, the findings and discussion in this paper add to the body of knowledge concerning the nature of tool-mediated work using flow techniques.

Acknowledgments

This work was supported with the financial support of the Science Foundation Ireland grant 13/RC/2094 and co-funded under the European Regional Development Fund through the Southern & Eastern Regional Operational Programme to Lero - the Irish Software Research Centre (www.lero.ie).

Appendix A. Description of interviewee job titles

Interviewee job titles at Company A	Job description	Interviewee job titles at Company B	Job description
(1) IT director	Responsible for overseeing the global manufacturing of IT activities.	(1) Network program flow manager	As part of the network management system is responsible for managing a number of software development teams and is a member of the senior management team within the company. Leading the steering group for the assimilation of flow practices within the company.
(1) Portfolio manager	Works with several project and program managers that would be responsible for delivering on several projects throughout the global organization.	(1) Network program flow & visualisation manager	Traditionally known in the company as Scrum Master. New title includes responsibility for managing a single project within the network program and as a member of the steer group assists the Network Program Flow Manager with the assimilation of flow practices.
(1) Systems integration manager	Responsible for managing the Oracle and SQL Production DBA teams, the Middleware support team and the Monitoring Solutions team.	(1) Network integration manager	Responsible for planning and managing the integration group, which focuses on integrating the different areas of the company that are delivering software.
(1) IT delivery manager	On a day-to-day basis it is essentially a Scrum Master role with extra responsibilities. This role was traditionally known in the company as IT Project Manager.	(1) Network program area manager	Responsible for managing 30 development teams across 6 sites and reports to the Network Program Flow Manager.
(1) Technical program manager	Focuses on the daily management of a software development team and reports to a Programme Manager who is responsible for managing a broader remit of activities.	(1) Network program manager	Responsible for replacing legacy technologies and with a new network management system and bring them together onto one extendable extensible platform.
(1) Financial controller	Provides an end-to-end overview of the finance performance of the business/IT units, ranging from contract appraisal through to performance of customer contracts and assisting with flow metrics such as Cost of Delay.	(1) Product development unit manager	As part of the Operations and Programs department which is an overall central department for all programs and product development in Ireland.
(1) Senior Business Analyst	Responsible for documenting the business and IT processes and assessing integration issues.	(1) Product development department manager	As Head of the Product development department is responsible for managing 70 software development teams that work on two products. Is also a member of the steering group responsible for the assimilation of flow practices.
(1) Software development consultant	Internal consultant responsible for EMEA (Europe, Middle East & Africa) Release Automation & DevOps.		

Appendix B. Classification of data using AT constructs

	A	B	C	D	E	F
	Subject (tension with tool)	Subject (tension with community)	Community (tension with tools)	Community (tension with subject)	Division of Labour	Rules & Norms
1	I wanted to tear it down, it showed how much work we had done and the amount that is ahead of us and if anybody was looking at that slide, they would pull out their hair and say this is impossible so I think you need to still create an environment where people think they can succeed.	If you have to tell the team that you have to finish that piece of testing before starting the next thing, they might say, well we can only have one guy doing it and the rest of us are going to hang around help him do it as fast as possible but it will still take 3 weeks and we are not allowed start the next thing because we have this VIP limit at a macro level. If you put a rule like that with a 100 team project, they are a relatively blunt instrument and then you have these micro impacts around your programme.	At the start people were saying it is in Jira, why would we need an illustration on the wall, this is something we can't maintain, it doesn't make sense.	The more concern that I would have is when you standardise something like flow and we talk about metrics and they are all good, the concern I would have is that it becomes a top down process that teams have to start adhering too. I would be a big believer in self-organising and empowered teams. I think it is important that, top down help can become a constraint for a software team and if we start putting in constraints that you must do this and you must do that, that can disempower the teams and take away some of their autonomy and impact performance.	The board are highest level requirements visualisation board. They give a certain view of the program at the highest level of the different development phases we go through but that main requirement is broken down into 2 further levels, epics and stories, where there's another level of visualisation required and that's not there at the moment, we're only visualising effectively end to end the high level requirements we have... that is the biggest difficulty is, setting out the roles, responsibilities and the process, the ways of working around that so people are actually aligned with producing stuff.	In information development, as I call it, we're transforming information from one format to another, it's invisible, so nobody can see what's impeding them and sometimes after a while because we're a large organisation there's kind of a culture of acceptance saying it's ok, it's not a big problem, we'll move on. So it's raising the awareness of flow and the culture of improvement I would say has to be raised, I think that's an impediment at the moment.
2	You have the conflict between your traditional project planning and your flow through the system. You can study all you like but it is going into a clocked pipe, no point pushing more in and that would have been a mindset before in that in the past you would have a team working on requirement X but we would say requirement Y needs to be delivered in this release so we better start with Y, and it typically needs 4 months development and there was a feeling that once we started it now we have a chance of making it. That would have been the typical clock the pipe behaviour. We had a lot of that and still have a tendency to do that.	No matter how hard you push the teams, teams are mainly developers and they are adverse to people tracking them and saying is your jira updated, is your story updated, did you move this, did you move that, you can move that, there is a dependency on this. There is an element of micro managing, we have our micro manage Jira, we have no choice at this stage.	What's up on the board should be in Jira and I probably have the view that we're trying to get the same data in 2 different places, on the board and electronically and that's a challenge as well. Ideally you would be able to get it all electronically and be able to visualise it then on a big screen or whatever we want to do but to me there's a huge amount of work in the manual maintaining of that board	I would say the majority of team members aren't looking at it, they're not looking at the board, it's only really some of the more top end managers are looking at the status up there, the area managers are looking at this, the majority of teams aren't looking at it at all. The teams are very focused on what they themselves have to do in the current sprint, that's what they're looking at.	Ultimately the people that really need to be focusing on this, probably the product owners and the project managers really need to understand that and get a grasp of it and teams maybe less so because they're working on their epics and user stories, the main requirement is the overall parent so that's where I see the division of people that need to be pulled into this.	There would definitely have been some cynics who would have said oh no, everybody needs to be working on something and we need to know what those things are and they need to work as hard as they possibly can. The idea of leaving guys idle waiting to solve problems would have been a huge cultural, mental challenge for some people in this organisation.
3						

References

- Ahmad, M.O., Markkula, J., Oivo, M., 2013. Kanban in software development: a systematic literature review. In: 2013 39th Euromicro Conference on Software Engineering and Advanced Applications. IEEE, pp. 9–16.
- Al Sarraj, Z.M., 1990. Formal development of line-of-balance technique. *J. Constr. Eng. Manag.* 116, 689–704.
- Allen, D.K., Brown, A., Karanasios, S., et al., 2013. How should technology-mediated organizational change be explained? A comparison of the contributions of critical realism and activity theory. *MIS Q.* 37, 835–854.
- Allen, D.K., Karanasios, S., Norman, A., 2014. Information sharing and interoperability: the case of major incident management. *Eur. J. Inf. Syst.* 23, 418–432.
- Anderson, D., 2013. Lean software development. Lean Kanban University (LKU), Seattle.
- Anderson, D., Concas, G., Lunesu, M.I., et al., 2011. Studying lean-kanban approach using software process simulation. In: *Agile Processes in Software Engineering and Extreme Programming*. Springer, pp. 12–26.
- Anderson, D.J., 2010. The Kanban Principles. Blue Hole Press.
- Arditi, D., Tokdemir, O.B., Suh, K., 2001. Effect of learning on line-of-balance scheduling. *Int. J. Project Manag.* 19, 265–277.
- Arditi, D., Tokdemir, O.B., Suh, K., 2002. Challenges in line-of-balance scheduling. *J. Constr. Eng. Manag.* 128, 545–556.
- Bang, S.K., Chung, S., Choh, Y., et al., 2013. A grounded theory analysis of modern web applications: knowledge, skills, and abilities for DevOps. In: *Proceedings of the 2nd annual conference on Research in information technology*. ACM, pp. 61–62.
- Benbasat, I., Goldstein, D.K., Mead, M., 1987. The case research strategy in studies of information systems. *MIS Q.* 369–386.
- Bertelsen, O.W., Bødker, S., 2000. Introduction: information technology in human activity. *Scand. J. Inf. Syst.* 12, 3.
- Birkeland, J.O., 2010. From a timebox tangle to a more flexible flow. In: *International Conference on Agile Software Development*. Springer, pp. 325–334.
- Canary, H.E., McPhee, R.D., 2009. The mediation of policy knowledge: an interpretive analysis of intersecting activity systems. *Manag. Commun. Q.* 23, 147–187.
- Capper, P., Williams, B., 2004. Enhancing evaluation using systems concepts. American Evaluation Association.
- Chen, R., Sharman, R., Rao, H.R., et al., 2013. Data model development for fire related extreme Events: an activity theory approach. *MIS Q.* 37, 125–147.
- Cocco, L., Mannaro, K., Concas, G., et al., 2011. Simulating kanban and scrum vs. waterfall with system dynamics. In: *International Conference on Agile Software Development*. Springer, pp. 117–131.
- Cole, M., Engeström, Y., 1993. A cultural-historical approach to distributed cognition. *Distrib. Cognit.* 1–46.
- Conboy, K., 2009. Agility from first principles: reconstructing the concept of agility in information systems development. *Inf. Syst. Res.* 20, 329–354.
- Ditsa, G., 2003. Activity theory as a theoretical foundation for information systems research. *Inf. Manag.* 192–231.
- Ebert, C., Abrahamsson, P., Oza, N., 2012. Lean software development. *IEEE Softw.* 22–25.
- Engeström, Y., 1987. Learning by expanding. Orienta-Konsultit Oy, Helsinki.
- Engeström, Y., 1993. Developmental studies of work as a testbench of activity theory: the case of primary care medical practice. *Understand. Pract.* 64–103.
- Engeström, Y., 2005. Developmental work research: Expanding activity theory in practice. Lehmanns Media.
- Erdogmus, H., 2010. Tracking progress through earned value. *IEEE Softw.* 27, 2–7.
- Fitzgerald, B., Russo, N.L., Stolterman, E., 2002. Information systems development: methods in action. McGraw-Hill Education.
- Fitzgerald, B., Stol, K.-J., 2014. Continuous software engineering and beyond: trends and challenges. In: *Proceedings of the 1st International Workshop on Rapid Continuous Software Engineering*. ACM, pp. 1–9.
- Fitzgerald, B., Stol, K.-J., 2015. Continuous software engineering: a roadmap and agenda. *J. Syst. Softw.*
- Foot, K.A., 2001. Cultural-historical activity theory as practice theory: illuminating the development of conflict-monitoring network. *Commun. Theory* 11, 56–83.
- Gallivan, M.J., 2001. Organizational adoption and assimilation of complex technological innovations: development and application of a new framework. *ACM SIGMIS Database* 32, 51–85.
- Greaves K. (2011) Taming the customer support queue.
- Hasan, H., 1998. Activity theory: a basis for the contextual study of information systems in organisations. *Inf. Syst. Act. Theory* 19–38.
- Hasan, H.M., Kazluaskas, A., Crawford, K.P., 2010. Blending complexity and activity frameworks for a broader and deeper understanding of IS. *Thirty First International Conference on Information Systems (ICIS)*. St. Louis.
- Helle, M., 2000. Disturbances and contradictions as tools for understanding work in the newsroom. *Scand. J. Inf. Syst.* 12, 7.
- Ikonen, M., 2010. Leadership in kanban software development projects: a quasi-controlled experiment. In: *Lean Enterprise Software and Systems*. Springer, pp. 85–98.
- Ikonen, M., Kettunen, P., Oza, N., et al., 2010. Exploring the sources of waste in kanban software development projects. In: 2010 36th EUROMICRO Conference on Software Engineering and Advanced Applications. IEEE, pp. 376–381.
- Ikonen, M., Pirinen, E., Fagerholm, F., et al., 2011. On the impact of kanban on software project work: an empirical case study investigation. In: *Engineering of Complex Computer Systems (ICECCS)*, 2011 16th IEEE International Conference on. IEEE, pp. 305–314.
- Ilyenkov, E., 1974. *Dialectical Logic: Essays on its History and Theory*. Moscow. Progress Publishers, Pacifica, CA, USA.
- Kaptein, V., 1996. Activity theory: implications for human-computer interaction. *Context consciousness* 103–116.

- Karanasios, S., Allen, D., 2014. Mobile technology in mobile work: contradictions and congruencies in activity systems. *Eur. J. Inf. Systems* 23, 529–542.
- Khurum, M., Petersen, K., Gorschek, T., 2014. Extending value stream mapping through waste definition beyond customer perspective. *J. Softw.* 26, 1074–1105.
- Kinoshita, F., 2008. Practices of an agile team. *Agile 2008 Conference*.
- Kitchenham, B.A., Pfleeger, S.L., Pickard, L.M., et al., 2002. Preliminary guidelines for empirical research in software engineering. *IEEE Trans. Softw. Eng.* 28, 721–734.
- Kniberg, H., Skarin, M., 2010. Kanban and Scrum-Making the most of both. Lulu.com.
- Korpela, M., Mursu, A., Soriyan, H.A., 2001. Information systems development as an activity. *Comput. Supported Cooperative Work (CSCW)* 11, 111–128.
- Kuutti, K., 1995. Activity theory as a potential framework for human-computer interaction research. In: *Context and consciousness: Activity theory and human-computer interaction*. MIT Press, pp. 17–44.
- Kuutti, K., Molin-Juustila, T., 1998. Information system support for 'Loose'Co-ordination in a network Organisation: an activity theory perspective. *Inf. Syst. Activity Theory* 73–92.
- Lee, A.S., 2001. Editor's comments: research in information systems: what we haven't learned. *MIS Q.* 25 V.
- Lyytinen, K., Rose, G.M., 2006. Information system development agility as organizational learning. *Eur. J. Inf. Syst.* 15, 183–199.
- Miles, M.B., Huberman, A.M., 1984. *Qualitative data analysis*. Sage, Beverly Hills.
- Miles, M.B., Huberman, A.M., 1994. *Qualitative data analysis: an expanded sourcebook*. SAGE publications, Inc.
- Mujtaba, S., Feldt, R., Petersen, K., 2010. Waste and lead time reduction in a software product customization process with value stream maps. In: *2010 21st Australian software engineering conference*. IEEE, pp. 139–148.
- Murphy, E., Manzanares, M.A.R., 2008. Contradictions between the virtual and physical high school classroom: a third-generation activity theory perspective. *Br. J. Educ. Technol.* 39, 1061–1072.
- Mursu, A.S., Luukkonen, I., Toivanen, M., et al., 2007. Activity theory in information systems research and practice: theoretical underpinnings for an information systems development model. *Inf. Res.* 12, 3.
- Nardi, B.A., 1996. Activity theory and human-computer interaction. *Context Consciousness* 7–16.
- Naylor, J.B., Naim, M.M., Berry, D., 1999. Leagility: integrating the lean and agile manufacturing paradigms in the total supply chain. *Int. J. Prod. Econ.* 62, 107–118.
- Nelson C.P. (2002) Contradictions in learning to write in a second language classroom: insights from radical constructivism, activity theory, and complexity theory.
- Nikitina, N., Kajko-Mattsson, M., 2011. Developer-driven big-bang process transition from scrum to kanban. In: *Proceedings of the 2011 International Conference on Software and Systems Process*. ACM, pp. 159–168.
- Nord, R.L., Ozkaya, I., Sangwan, R.S., 2012. Making architecture visible to improve flow management in lean software development. *Softw., IEEE* 29, 33–39.
- Ohno, T., 1988. *Toyota production system: beyond large-scale production*. crc Press.
- Olsson, H.H., Bosch, J., 2014. Climbing the "Stairway to Heaven": evolving from agile development to continuous deployment of software. In: *Continuous Software Engineering*. Springer, pp. 15–27.
- Oppenheim A. (1992) Questionnaire design, interviewing and attitude measurement. Continuum.
- Packer, M.J., Goicoechea, J., 2000. Sociocultural and constructivist theories of learning: ontology, not just epistemology. *Educ. Psychol.* 35, 227–241.
- Petersen, K., Roos, P., Nyström, S., et al., 2014. Early identification of bottlenecks in very large scale system of systems software development. *J. Softw.* 26, 1150–1171.
- Petersen, K., Wohlin, C., 2009. Context in industrial software engineering research. In: *Proceedings of the 2009 3rd International Symposium on Empirical Software Engineering and Measurement*. IEEE Computer Society, pp. 401–404.
- Petersen, K., Wohlin, C., 2011. Measuring the flow in lean software development. *Softw.* 41, 975–996.
- Polk, R., 2011. Agile and kanban in coordination. *AGILE* 263–268.
- Poppendieck, M., 2002. Principles of lean thinking. *IT Manag. Select* 18.
- Poppendieck, M., Cusumano, M.A., 2012. Lean software development: a tutorial. *Softw., IEEE* 29, 26–32.
- Poppendieck, M., Poppendieck, T., 2003. *Lean software development: an agile toolkit*. Addison-Wesley Professional.
- Poppendieck, M.B., Poppendieck, T., 2013. *The Lean Mindset: Ask the Right Questions*. Pearson Education.
- Power, K., 2014. Definition of ready: an experience report from teams at cisco. In: *Agile Processes in Software Engineering and Extreme Programming*. Springer, pp. 312–319.
- Power, K., Conboy, K., 2015. A metric-based approach to managing architecture-related impediments in product development Flow: an industry case study from cisco. In: *Software Architecture and Metrics (SAM), 2015 IEEE/ACM 2nd International Workshop on*. IEEE, pp. 15–21.
- Reinertsen, D.G., 2009. *The principles of product development flow: second generation lean product development*. Celeritas Redondo Beach, Canada.
- Rubin, H., Rubin, I., 2005. *Qualitative interviewing: The art of hearing data*. Sage, Thousand Oaks, CA.
- Runeson, P., Höst, M., 2009. Guidelines for conducting and reporting case study research in software engineering. *Emp. Softw. Eng.* 14, 131–164.
- Rutherford, K., Shannon, P., Judson, C., et al., 2010. From chaos to kanban, via scrum. In: *International Conference on Agile Software Development*. Springer, pp. 344–352.
- Schatzki, T.R., 1998. *Social practices: A Wittgensteinian approach to human activity and the social*. Cambridge Univ Press.
- Senapathi, M., Middleton, P., Evans, G., 2011. Factors affecting effectiveness of agile usage—insights from the BBC worldwide case study. In: *International Conference on Agile Software Development*. Springer, pp. 132–145.
- Shinkle, C.M., 2009. Applying the dreyfus model of skill acquisition to the adoption of kanban systems at software engineering professionals (SEP). In: *Agile Conference, 2009. AGILE'09*. IEEE, pp. 186–191.
- Sjøberg, D.I., Johnsen, A., Solberg, J., 2012. Quantifying the effect of using kanban versus scrum: a case study. *IEEE Softw.* 29, 47–53.
- Spasser, M.A., 2002. Realist activity theory for digital library evaluation: conceptual framework and case study. *Comput. Supported Cooper. Work (CSCW)* 11, 81–110.
- Stake, R., 2000. Case Studies' in *Handbook of Qualitative Research*, Denzin, N. and Y. Sage, Thousand Oaks.
- Staron, M., Meding, W., 2011. Monitoring bottlenecks in agile and lean software development projects—a method and its industrial use. In: *International Conference on Product Focused Software Process Improvement*. Springer, pp. 3–16.
- Stray, V.G., Moe, N.B., Dingsøyr, T., 2011. Challenges to teamwork: a multiple case study of two agile teams. In: *International Conference on Agile Software Development*. Springer, pp. 146–161.
- Tichy, M., Bosch, J., Goedicke, M., et al., 2015. 2nd international workshop on rapid continuous software engineering (RCoSE 2015). In: *Proceedings of the 37th International Conference on Software Engineering—Volume 2*. IEEE Press, pp. 993–994.
- Towill, D., Christopher, M., 2002. The supply chain strategy conundrum: to be lean or agile or to be lean and agile? *Int. J. Logist.* 5, 299–309.
- Vermeulen, H., Gain, J., Marais, P., et al., 2016. Reimagining gamification through the lens of activity theory. *49th Hawaii International Conference on System Sciences (HICSS)*.
- Vygotsky, L., 1978. Interaction between learning and development. *Readings on the development of children* 23, 34–41.
- Wang, X., Conboy, K., Cawley, O., 2012. "Leagile" software development: an experience report analysis of the application of lean approaches in agile software development. *J. Syst. Softw.* 85, 1287–1299.
- Wengraf, T., 2001. *Qualitative research interviewing: Biographic narrative and semi-structured methods*. Sage.
- White, L., Burger, K., Yearworth, M., 2016. Understanding behaviour in problem structuring methods interventions with activity theory. *Eur. J. Oper. Res.* 249, 983–1004.
- Wohlin, C., Runeson, P., Höst, M., et al., 2012. *Experimentation in software engineering*. Springer Science & Business Media.
- Womack, J.P., Jones, D.T., 2010. *Lean thinking: banish waste and create wealth in your corporation*. Simon and Schuster.
- Womack, J.P., Jones, D.T., Roos, D., 1990. *How lean production can change the world*. New York Times Mag. 23, 20–38.
- Yin, R.K., 2003. *Applications of case study research*. Sage Publications, Inc.
- Yin, R.K., 2009. *Case Study Research: Design and Methods*, 4. Sage Publications, udgave.

Denis Dennehy is a Postdoctoral Researcher with the Lero Irish Software Research Centre at National University of Ireland Galway, Ireland. Prior to joining Lero, he completed his doctoral research at the Business Information Systems department, University College Cork (UCC), Cork, Ireland. During his time at UCC he received the InterTradeIreland *Marion McAneney Graduate Research Award*. Denis has consulted on a number of industry research projects funded by Enterprise Ireland, a government organisation responsible for the development and growth of Irish enterprises in world markets. He has presented his research at international conferences and published in international journals.

Kieran Conboy is a Professor in Information Systems with the Lero Irish Software Research Centre at National University of Ireland Galway, Ireland. He previously worked for Accenture Consulting and the University of New South Wales in Australia. He has worked with organisations such as Microsoft, Atlassian, Cisco Systems, Suncorp, and Fidelity Investments, as well as many SMEs. Kieran has published over 100 articles in leading international journals and conferences including Information Systems Research, the European Journal of Information Systems, and the Journal of the AIS. He is an editor of the European Journal of Information Systems and has chaired many international conferences in his field.